

Core Math Diff Audit



June 15, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Median Computation	5
Power-of-Ten Detection	5
Security Model and Trust Assumptions	6
Notes & Additional Information	7
N-01 Potential Overflow in the quick_sort_by! Midpoint Calculation	7
N-02 Inaccurate and Incomplete Documentation on Sorting and Median Behavior	7
N-03 Style and Convention Inconsistencies Across the Math Package	8
N-04 median! Widens Entire Input to vector<u256>, Increasing Gas and Memory Use	10
Conclusion	11
Appendix	12
Issue Classification	12

Summary

Type	Library	Total Issues	4 (3 resolved, 1 partially resolved)
Timeline	From 2026-06-03 To 2026-06-05	Critical Severity Issues	0 (0 resolved)
Languages	Move	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	0 (0 resolved)
		Notes & Additional Information	4 (3 resolved, 1 partially resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin conducted a [diff audit](#) of the [OpenZeppelin/contracts-sui](#) repository, covering the changes to the `openzeppelin_math` package between base commit [af09be7](#) and target commit [11a0c2e](#).

In scope were the following modified or added files:

```
math
├── core
│   └── sources
│       ├── internal
│       │   ├── common.move
│       │   └── macros.move
│       ├── decimal_scaling.move
│       ├── rounding.move
│       ├── u8.move
│       ├── u16.move
│       ├── u32.move
│       ├── u64.move
│       ├── u128.move
│       ├── u256.move
│       ├── u512.move
│       └── vector.move
```

System Overview

The `openzeppelin_math` package provides general-purpose unsigned-integer math primitives for Sui Move modules, exposing per-width helpers for operations such as `mul_div`, logarithms, square roots, and modular inverse, together with a 512-bit helper type used to absorb intermediate overflow. The audit scope is a diff that introduces a new median computation surface to the `vector` module, replaces the `u128` and `u256` powers-of-ten detection with a macro-based implementation, and rewrites same-module calls in `u512` (including the `div_rem_u256` long-division routine) to receiver-call syntax as a semantic no-op, alongside the relocation of constants, the removal of an unused import, and comment- and documentation-level changes across the remaining files in scope.

Median Computation

The `vector` module now exposes a generic `median!` macro and a concrete `median_u256` function that compute the median of an unsigned-integer vector. The macro widens the input to a `u256` working copy and delegates to `median_u256`, which consumes its argument and routes through a quickselect kernel with median-of-three pivot selection and three-way (Dutch-flag) partitioning. Odd-length inputs return the central order statistic directly; even-length inputs combine the two central values through `macros::average!` under a caller-supplied `RoundingMode`.

The median computation flow inside `median_u256` proceeds as follows:

- An empty input aborts deterministically with `EMedianOfEmptyVector`.
- For odd-length input, `select_k_u256(&mut vec, mid)` returns the median directly.
- For even-length input, `select_k_u256(&mut vec, mid)` returns the upper central value, `max_prefix_u256(&vec, mid)` returns the lower central value as the maximum of the partition prefix, and `macros::average!` merges them under the requested rounding mode.

Power-of-Ten Detection

The wide-width public functions `u128::is_power_of_ten` and `u256::is_power_of_ten` now delegate to a new package-scoped `is_power_of_ten!`

macro, replacing the previous lookup-table implementation. The macro tests an input `n` by computing the candidate exponent via the existing `log10_floor` helper and comparing `n` against `10u256.pow(exp)`. The `u8`, `u16`, `u32`, and `u64` versions are not rewired: they still check the input against a fixed list of powers of ten directly.

Security Model and Trust Assumptions

The `openzeppelin_math` package is a pure library: it holds no on-chain state, owns no objects, and exposes no privileged callers, capability gates, or value-routing functions. Correctness therefore reduces to the numerical accuracy of each routine and the integrity of the Move runtime guarantees on which the routines depend. Integrators are assumed to perform their own input validation and to enforce any access control at their own call sites before delegating to the math primitives.

The audit relies on the following trust assumptions:

- The Sui Move VM aborts deterministically on integer overflow, underflow, and out-of-bounds vector access; the package depends on this default behavior rather than introducing redundant range checks at each step.
- The Sui standard-library helpers used by the package (notably `std::u256::pow`, the `vector` builtins, and the integer cast operators) are assumed to be correct and to abort on overflow.
- The package exposes no privileged role. Every public function, including the new `median!` and `median_u256` entry points, the package-scoped `is_power_of_ten!` macro, and the `u512::div_rem_u256` helper, is callable by any caller of the package without admin, owner, or capability checks.

Notes & Additional Information

N-01 Potential Overflow in the `quick_sort_by!` Midpoint Calculation

The `quick_sort_by!` macro selects a partition pivot using a median-of-three strategy. It computes the midpoint of the active range as $(start + end) / 2$. This expression overflows `u64` when `start + end` exceeds the maximum `u64` value.

The condition is not reachable in practice, as it would require a vector of approximately 2^{63} elements, which cannot be allocated within a transaction. The concern is therefore one of consistency and defense in depth rather than exploitability. Notably, the sibling routine `select_k_u256` already avoids the pattern by computing the midpoint as $start + (end - start) / 2$, which cannot overflow.

Consider applying the same overflow-safe form, $start + (end - start) / 2$, in `quick_sort_by!`, so that both routines share a single robust midpoint computation.

Update: Resolved in [pull request #357](#). The OpenZeppelin development team stated:

| Suggestion have been applied.

N-02 Inaccurate and Incomplete Documentation on Sorting and Median Behavior

The `vector` module exposes the package's sorting and median surface through the `quick_sort!` and `quick_sort_by!` macros, the `median!` macro, and the concrete `median_u256` function. The accompanying doc comments are the contract that integrators rely on before adopting these functions, so they should accurately reflect each function's behavior, complexity, and side effects. Several inaccuracies and omissions were identified:

- The fenced `median!` and `median_u256` examples open with `use openzeppelin_math::rounding` without a trailing semicolon, so the published snippets do not parse as Move source. The examples are also internally inconsistent: the

`quick_sort!` and `quick_sort_by!` examples omit `use` statements entirely, while the median examples list them as two single-item lines rather than the grouped form `use openzeppelin_math::{rounding, vector};`.

- The documented complexity is inconsistent and, in places, overstated. The `quick_sort_by!` doc comment discloses the $O(n^2)$ theoretical worst case, but the `median!` and `median_u256` doc comments omit it entirely, even though they delegate to the same deterministic median-of-three quickselect. At the same time, both sort macros overstate the guarantee, describing the worst case as "much less likely for common inputs" and the macro as "suitable for arbitrarily large vectors". The quadratic worst case remains reachable for adversarially shaped inputs, which is relevant whenever an integrator passes attacker-controlled vectors into the surface.
- The comparator contract is underspecified. The `$le` documentation describes only the performance consequence of a poor comparator. It does not state that an inconsistent, non-transitive, or non-total comparator silently yields an unsorted result without aborting, which is a correctness concern rather than a performance one. It also does not state that `$le` is invoked up to twice per probe, since a second call is issued in the equality branch of the `three-way partition` when the first call returns `true`. Integrators passing an expensive comparator may therefore underestimate the resulting gas cost.

Consider correcting the example blocks so that they parse and adopt a single import convention, adding an accurate $O(n^2)$ worst-case note and a caution against unbounded, attacker-controlled inputs to all four entry points, and amending the comparator contract to document both its validity requirement (a consistent total preorder) and its per-probe invocation count.

Update: Partially Resolved in [pull request #359](#). The OpenZeppelin development team stated:

Suggestion applied except for grouping vector and rounding imports as the formatter behavior is to separate them.

N-03 Style and Convention Inconsistencies Across the Math Package

Several cosmetic and convention-level inconsistencies were identified across the math package. None affects correctness, but each represents a small drift that future maintainers would otherwise need to rediscover from context:

- Section headers do not follow a single convention. In `macros.move`, the casing is mixed: `// === Package Functions ===` is in title case, while `// === Helper`

`functions ===` and `// === Internal helpers for modular arithmetic ===` are in sentence case. In `vector.move`, `// === Internal Functions ===` departs from the `// === Private Functions ===` convention used in `u512.move` and `decimal_scaling.move`, and `// === Concrete Public Functions ===` appears in no other file. As a discretionary point, `rounding.move` uses `// === Structs ===` for the `RoundingMode` enum; grouping enums under a `Structs` section is an accepted convention, so this last item is optional.

- The `BIT_WIDTH` constant carries a docstring only in `u256.move`. The same constant in `u8.move`, `u16.move`, `u32.move`, `u64.move`, and `u128.move` is undocumented.
- The constant `MAX_LOG_10` reads on first encounter as "log base 10 of 10" rather than "maximum value of `log10`". A name such as `MAX_LOG10` would be unambiguous.
- The refactor that replaced the `is_power_of_ten` lookup tables with a `log10_floor + pow` derivation left two doc comments stale. The public `is_power_of_ten` docstrings on `u128.move` and `u256.move` no longer describe the implementation strategy, and the `log10_floor` comment still states "binary search over powers of 10" although the body is now a digit-magnitude division cascade.
- Several non-obvious code paths in `vector.move` lack clarifying comments:
- The `insertion-sort base case` derives strict-less-than from the non-strict `$le` via `$le(a,b) && !$le(b,a)`, a standard but unexplained idiom.
- The `three-way partition and pivot placement` issue harmless `vec.swap(i, i)` calls on the live paths, for example when the less-than region is empty or when no element exceeds the pivot. Sui's `vector::swap` treats equal indices as a no-op, but the call sites do not note that this is intentional.
- The `partition region comments` label the equal region without explaining why the pivot-equivalent elements end up contiguous after the final swap.
- The odd-length "rounding has no effect" guarantee is documented on `median!` but is not reinforced at the `corresponding branch` of `median_u256`.
- The `0(log n)` stack-depth bound holds only because the larger partition is pushed first and the smaller is processed first; the `push-ordering branch` is therefore load-bearing, but the comment does not note this.

Consider applying a single style-and-consistency pass: align the section-header naming and casing across `rounding.move`, `macros.move`, and `vector.move`; document `BIT_WIDTH` uniformly across the width-specific modules; rename `MAX_LOG_10` to `MAX_LOG10`; refresh the `is_power_of_ten` and `log10_floor` documentation to match the implementation introduced in pull request #323; and add the short clarifying comments noted above, so that the package presents a single, predictable style to maintainers and integrators.

Update: Resolved in [pull request #360](#). The OpenZeppelin development team stated:

| *Improved comments consistency from suggestions.*

N-04 `median!` Widens Entire Input to `vector<u256>`, Increasing Gas and Memory Use

The `median!` macro computes a median for unsigned integer vectors by delegating to `median_u256`, which consumes and partially reorders a `vector<u256>` through quickselect. Because the input is borrowed and the selection routine requires an owned, reorderable buffer, `median!` first converts the entire input into a `vector<u256>` via `vec.map_ref!(|x| *x as u256)`.

This conversion is unconditional and scales linearly with the input length, allocating and populating a full same-length `vector<u256>` before any selection occurs. For narrow element types the working buffer is much larger than the data it represents, up to thirty-two times for a `vector<u8>`, which raises the peak memory and gas cost of the operation. The effect is a constant factor and does not alter the asymptotic cost.

Consider performing the selection at the input width and widening only the one or two selected central values before averaging, which would reduce the working-buffer cost for narrow types. This entails generalizing the selection routine, which currently centralizes its logic in a single `u256` implementation, so the trade-off between that centralization and the gas saving should be weighed. Consider also documenting an expected input bound, so that integrators avoid invoking the median surface on unbounded, attacker-influenced vectors.

Update: Resolved in [pull request #363](#). The OpenZeppelin development team stated:

| *Refactored and improved the performance significantly.*

Conclusion

The `openzeppelin_math` package provides pure, stateless unsigned-integer arithmetic primitives that other Sui Move packages build upon. OpenZeppelin audited the in-scope diff, which adds a median computation surface to the `vector` module, reimplements the `u128` and `u256` powers-of-ten checks on top of a shared macro, and applies a set of behavior-preserving refactors across the remaining modules.

The new median logic and the reworked powers-of-ten checks behaved as intended throughout the review, and no defects were identified in the numerical results of the in-scope routines during the engagement. Because the package holds no state and exposes no privileged roles, its security reduces to the correctness of each computation and to the clarity of the contracts that integrators rely on. The issues identified were limited in severity and concentrated in those contracts: several doc comments did not fully reflect the behavior, complexity, and side effects of the functions they describe, and a number of code-convention inconsistencies were observed across the package.

The most consequential of these concern integrator expectations. The sorting and median routines carry a quadratic worst-case cost on adversarially ordered inputs, and that property, together with the comparator and input-size contracts, is not consistently documented at the surfaces integrators read. Documenting these contracts accurately, so that integrators bound untrusted input and supply valid comparators, would reduce the risk of misuse. The remaining observations are low-impact robustness and efficiency refinements that do not affect the correctness of the library as implemented.

We thank the OpenZeppelin development team for their responsiveness and willingness to provide technical context throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.